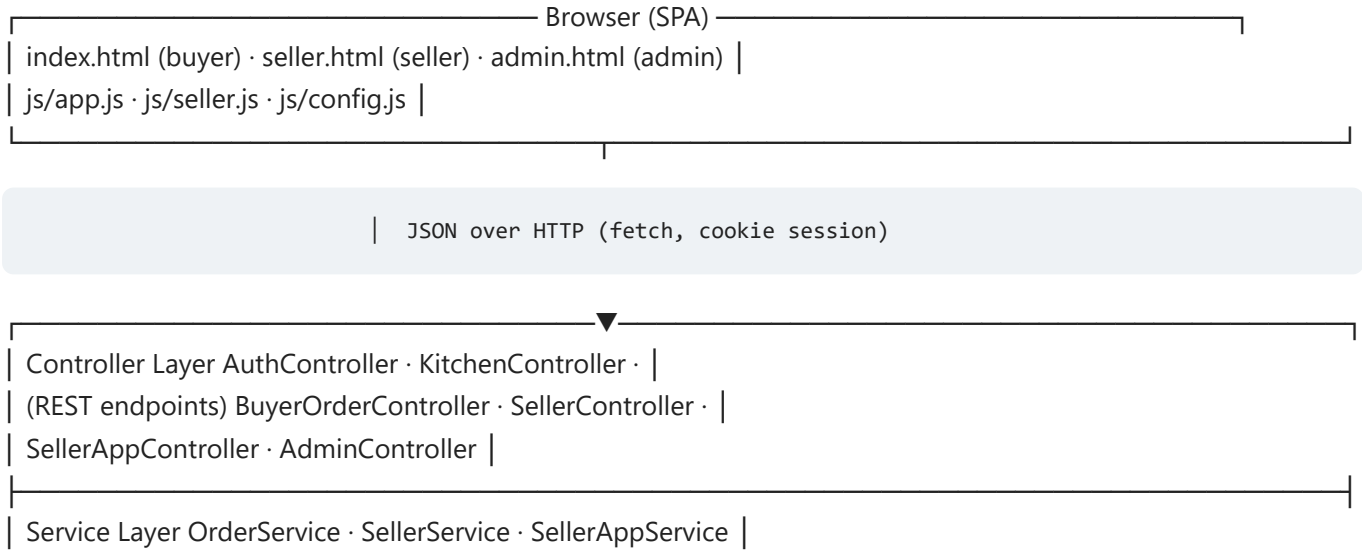
SocioMart Technical Document

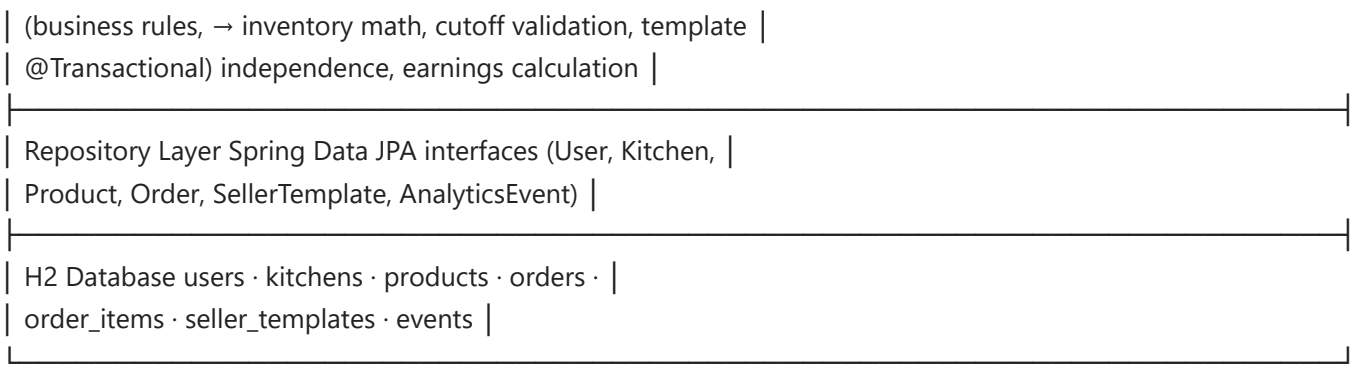
1. Technology Stack

Layer	Technology	Why It Was Chosen
Language	Java 21	Stable LTS, strong typing, industry standard for enterprise APIs
Framework	Spring Boot 4.1.1	Fast REST development, built-in dependency injection, security, validation
Data Layer	Spring Data JPA + Hibernate	Object-relational mapping without boilerplate SQL; automatic schema from entities
Database	H2 (in-memory)	Relational integrity for orders/payments; reseeded with Indian food demo data on every boot
Build Tool	Maven (mvnw wrapper)	Reproducible builds — no local install needed, works in CI
Frontend	HTML5 + CSS3 + Vanilla JS (SPA)	Zero build step, instant load, served straight from Spring's static folder
Sessions	HttpSession cookies	Simple, secure login state for buyer/seller flows
Public Access	Render (Docker, free tier)	Free HTTPS public URL — single service serves both Buyer (/) and Seller (/seller.html) with shared H2 state
CI	GitHub Actions	Every push is auto-compiled so broken code can never reach main
Docs & Testing	Edge headless (PDF), PowerShell E2E harness	Screenshot-driven docs, regression suite

2. System Architecture

The application follows a classic 3-layer architecture. The browser never talks to the database directly — every request flows through the API layer, which enforces login and ownership rules.





Key design decision: GlobalExceptionHandler converts every business error (sold-out item, unauthorised seller, cap reached) into a clean JSON {error, message} response with the correct HTTP status — the frontend never crashes on an unexpected payload.

3. Database Design

Seven tables, one per real-world concept. Relationships use JPA ManyToOne joins; demo data is seeded by DemoDataSeeder on first boot.

Table	Purpose	Key Columns
users	Buyers, sellers, admin	mobile_number (unique), name, is_seller, flat_number
kitchens	A seller's shopfront	seller_id (FK), name, slug, rating, available_today, verified
products	One day's live offering	kitchen_id (FK), price, max_quantity, remaining_quantity, available_date, cutoff_time, ready_by_time
orders	One purchase event	buyer_id, kitchen_id, total_amount, payment_status, order_status, buyer_name, buyer_flat
order_items	Line items of an order	order_id (FK), product_id (FK), product_name, quantity, unit_price
seller_templates	Saved favourites (max 3)	seller_id (FK), name, price, max_quantity, cutoff_time
analytics_events	Raw behaviour log	kitchen_id, event_type (MENU_VIEW...), created_at

Why remaining_quantity is its own column: the stepper updates one integer per click instead of re-deriving stock from orders — fast and race-safe. Why order_items.product_name is denormalised: a dish can be renamed or deleted later; the historical order must still show exactly what the buyer ordered.

4. REST API Reference

All endpoints return JSON. Errors always arrive as { "error": "`", "message": "" } with a proper HTTP status (400 bad input, 401 not logged in, 403 wrong owner, 404 missing, 409 conflict).`

Method & Path	Who	What It Does
POST /api/auth/otp/request, verify	public	OTP login / registration, creates session
GET /api/kitchens, /{slug}	public	Kitchen list / one kitchen with today's offerings (logs MENU_VIEW)
POST /api/buyer/orders/draft	buyer	Price the cart server-side
POST /api/buyer/orders/place	buyer	Place order, decrement stock atomically
GET /api/seller-app/dashboard	seller	Earnings, offerings, orders, real view count
POST /api/seller-app/products	seller	Create offering (manual / quick-post confirm)
PATCH .../products/{id}/inventory	seller	Stepper: add/subtract stock by delta
POST .../products/{id}/sold-out	seller	Force mark sold out (works on unlimited items)
GET/POST/DELETE /api/seller-app/templates...	seller	Manage the 3 saved favourites
POST /api/seller-app/templates/{id}/publish	seller	Republish favourite as independent offering
POST /api/seller-app/parse-message	seller	WhatsApp Quick Post parser (read-only)
GET /api/seller-app/orders/summary	seller	Per-dish plates + revenue
GET /api/seller-app/orders/product/{id}	seller	Per-buyer drill-down
GET /api/seller-app/earnings	seller	Pending / confirmed-today / this-month
PATCH /api/seller/orders/{id}/status	seller	Order pipeline transitions (incl. cancel)
PUT /api/seller/products/{id}	seller	Partial update (price, cutoff, stock...)
/api/admin/...	admin	Dashboard, buyers, sellers, kitchens, offerings, orders, enquiries, analytics, seller approval

5. Security Model

- Role-based access: BUYER, SELLER, ADMIN
- Buyer endpoints: require BUYER role
- Seller endpoints: require SELLER role
- Admin endpoints: require ADMIN role
- Ownership checks on every mutation: seller B cannot modify seller A's kitchen/products/orders (HTTP 403)
- ID isolation: buyer cannot access another buyer's order (HTTP 404)
- Anonymous access to admin endpoints: rejected (HTTP 401)
- Session timeout: 30 minutes
- SameSite=Lax cookie protection

6. Testing

6.1 Unit Tests (JUnit 5)

13 tests across 3 test classes:

- OrderServicePaymentTest (5 tests): payment status -> order status mapping
- FavouriteServiceLimitTest (4 tests): 3-template cap enforcement
- OrderServiceValidationTest (4 tests): cutoff, stock, duplicate order validation

6.2 E2E Regression Suite

A PowerShell harness drives the real running application over HTTP — the same path a browser takes — and asserts behaviours. It is delta-based, so it can be re-run on a live instance any number of times without false failures.

6.3 Build Verification

- mvnw clean compile: PASS (97 source files, Java 21)
- mvnw test: 13/13 PASS
- mvnw package -DskipTests: PASS

7. Deployment

Hosting provider: Render.com — free Docker web service. The app is built from the existing Dockerfile (multi-stage Maven -> JRE 21) and deployed as a Render Blueprint via render.yaml. One service serves both Buyer (/) and Seller (/seller.html) with a shared in-process H2 database — no split instances, no data desync.

Render injects the PORT environment variable automatically; the app binds to ${PORT:8081}. Render auto-deploys on every push to main. The free tier sleeps after ~15 min of inactivity; a free uptime pinger keeps the service warm.

Public URLs (single Render service):

- https://sociomart-demo.onrender.com/ — Buyer view
- https://sociomart-demo.onrender.com/seller.html — Seller dashboard
- https://sociomart-demo.onrender.com/api/kitchens — health probe

8. Known Limitations

- H2 in-memory database: custom data disappears on restart; demo data re-seeds on every boot
- Free tier cold-start delay (~120s) on Render
- No real OTP delivery (dev mode returns fixed OTP)
- Mockito self-attaching warning on JDK 21 (non-fatal)